LLM Classification Fine-Tuning

Shalini P,Saraswathi N,Nitheesha M,T Purnima Krishna
Department of Computer Science and Engineering
Shiv Nadar University Chennai, India
Student IDs: 24011101076,24011101069,24011101068,24011101114
Emails: parvathareddyshalinireddya@gmail.com,nalamothusaraswathi@gmail.com,nitheeshamoturu@gmail.com,
tadelapurnima@gmail.com

The project we have selected out of many is Kaggle LLM classification Fine Tuning Competition.The main objective is to predict one good response that humans are likely to prefer out of two responses generated by two different AI models.This problem helps improve AI accuracy and efficiency in answering people in a way humans want.dataset kaggle has provided contains 57,477 training examples containing total of 9 columns.It is tested through which we knew that it is balanced and there are no missing values.The proposed method is a dual model where the output from the basic machine learning model and the deep learning are blended to detect various types of patterns in the dataset. We have extracted many numerical features including characters,words,paragraph counts,prompt length, log of difference between the lengths which are normalized to eliminate verbosity bias.Also we have included text character and word related frequency-inverse document frequency features which are sent as input to logistic regression so the model understands significant parts of the text.All the features extracted are sent to lightGBM for learning.Parallely we send raw text to the model LORA finetuned QWEN2.5 model.We have contributed in selecting,trying out different models along with trying out dual path of models which worked in our case.The final score we have achieved is log loss of 0.99361.

Code repository:
https://github.com/shaliniparvathareddy/
LLM-Classification-Finetuning

***Index Terms*—Human preference prediction, Fine-Tuning, LLM's, log-loss, Chatbot Arena**

I. Introduction

The problem that our competition tries to address is to answer the question which among the two responses user might prefer for a same prompt. why this matters : generally manually annotating takes more time and effort. so training a model to predict human preferences helps identify human preferences more easily. this model that we develop can be used while training chatbots and in customer support to train them according to human preferences. Objectives: the objective is to build a models that takes three inputs prompt, response A and response B (here the responses can from different llms and models) and predict the probabilities for each of these classes model a wins, model b wins or tie. Challenges:

1) the model may prefer first prefer first response because it appears in the beginning. this is called position bias.
2) different persons prefer different responses.
3) also sometimes the model may prefer a model with more length this is called verbosity bias.
4) sometimes two models may give answers with same meaning but with different words so the model must understand the meaning.

Our team implementation: we used two main approaches we extracted 218 features from the input and used LightGBM and Logisitic Regression model to understand statiscal features and to understand meaning we have used Qwen 2.5-1.5B with LoRA. Evaluation Metric : the evaluation metric is log loss. Rules/constraints:

1) maximum of 10 submissions in day.
2) notebook cannot run for more than 7 hours.
3) training dataset has 55,000 rows and testing dataset has 25,000 rows

Competition status: the currently is currently active. Contributions:

1) Extracted 218 features from the input.
2) Fine tuned Qwen 2.5-1.5B model to understand the semantic features.
3) Combined three models logistic regression+ TF-IDF, LightGBM and fined tuned Qwen 2.5-1.5B +LoRA.

Report Roadmap : The reports includes the following sections:

1) Related Works
2) Proposed methodology
3) Results and Discussion
4) Conclusion and future Work

II. Related Works

Three public solution from the competition:

1) Large LLm ensemble [1]: In this two models were used Gemma-2-9B and Llama-3-8B. both were fined tuned with LoRA. they also found out that increasing the content and swapping the order of testing

improved respones improved the performance. the log loss of this model was 0.88017

2) Small Transformer Model [2]:This solution uses DeBERTa-v3-xsmall. TF-IDF and response length were also included as features. the log loss of this model was 1.0583.
3) Traditional Machine learning approach [3]:This model created features like word counts, character counts, response lengths and differences between two responses. this solution also uses a machine learning model LighGBM with above created features. this solution also uses response swapping to reduce position bias. the log loss of this model was 1.020.

Comparision with our work: Our approach included creating around 218 features from the inputs and we have dual path one path has :

1) Logisitic regression on TF-IDF text features
2) LightGBM on TF-IDF text features and manually created 218 features

other path has:

1) Fine tuned Qwen 2.5-1.5B with LoRA for semantic models. Intially we used MiniLM instead of Qwen but since the context length of MiniLM. we shifted to Qwen.

the outputs from these models were combined and then we used calibration.

So our approach combines traditional machine learning with LLM finetuning. the log loss of our model was 0.99361.

TABLE I
COMPARISION BETWEEN OUR MODEL AND PUBLIC COMPETITION MODELS

Approach	**Model**	**Reported Log Loss**
Large LLM ensemble	Gemma-2-9B	0.88017
Small transformer	DeBERTa-v3-xsmall	1.0583
Traditional ML approach	LightGBM	1.020
Our Approach	Qwen2.5-1.5B+LightGBM+Logisitic Regression	0.99361

III. Proposed Methodology

A. Problem Formulation and Dataset

This is a Kaggle llm fine tuning classification problem.The main goal is to which responses given from multiple models is the best.The input will be prompt and two responses.The output will be response a or response b or tie.The dataset is the llm-classification from Kaggle,it consists of train,test and sample submission files.The dataset is a balanced dataset.The training datset has 57477 labels.Winner lbels are represented as one hot enoders because the task is involving three classes.This will help in better model training.The three outcomes have close percentages like model_a has 34.91 %,model_b has 34.19% and tie has 30.90% which clearly shows that dataset is not favouring any outcome.The dataset split is 80:20 for training and validation to avoid the data leakage.Stratified split is used for the target class.The test dataset will generate predictions for the submission and the labels will be hidden here.

B. Processing Pipeline and Approach

Firstly the training data,test data and submission files are taken from the Kaggle competition files.The training data has labelled examples whereas the test data will generate the predictions and the sample submission fike will show the output format.The rows in the dtaset represent the labelled examples whereas the columns represent the attributes like prompts and responses.No missing values are there in the dataset.The percentages for the outcomes are monitored so that we can check if the dataset is imbalanced or favours any outputs.Characteristics like prompt lengths are measured to analyse the text complexity and can be used in feature extraction.The data will require pasrsing so we used json parsing.This is used to transform the raw text into a data that the model processes.Aftwr this feature extraction is done i.e the data is converted into numerical inputs so that the useful information about the data is etracted.The length of prompt and responses are measured so that the final response is not biased towards the one with higher length.Text_quality features are calculated so that the response with higher number of repeated words and unnecessary punctuations are avoided.Other feature groups like prompt_response relationship,semantic features and bias_related features are calculated so that the model will be able to avoid self promotion and other type of biases.These numerical features are tjen provided to the classifer to predict the outputs.The model will predict the outcomes and then compare it with the correct labels.The validation dataset set will evaluate how well it will be generalized on the unseen data.Once the training is done,the model will preprocess on new test data which don't have any known labels.

C. Implementation Details

The programming language used is python and the environment is Kaggle notebook.80% of the training data is used to teach the model and 20% of the validation data is used to check the accuracy of the predictions.Stratification is used so that it preserves the exact proportion fo the target classes.GPU acceleration is used to process the mathematical problems much faster.The implementations in the notebook are data validation,EDA,parsing,preprocessing,splitting the data,feature engineering,logistic regression and

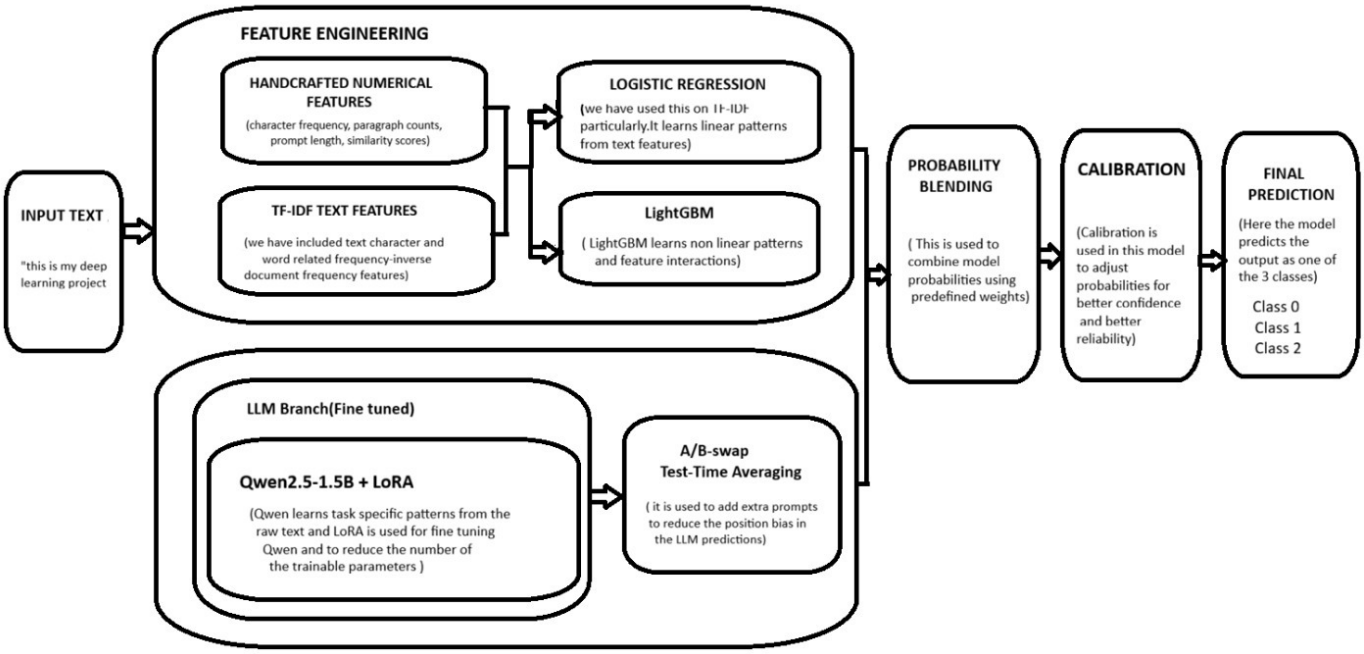


Fig. 1. Overall architecture of the proposed model.

lighGBM,qwen2.51.5B finetuning using LoRa,probability blending,calibration and final class prediction.

IV. Results and Discussion

A. Experimental Setup and Evaluation

The experimental setup we used is to evaluate the results are based on the human preferred response that were generated by two different LLM's which forms a 3 class classification problem.Our model at the end gives a probability distribution of the 3 possible classes. this is used because in our competition we want to see how accurately the model is predicting the probabilities rather than checking whether the predicted class is correct or not.This model was evaluated in multiple stages.

Validation Strategy: In this model we are using 3fold StratifiedGroupKFold cross validation to avoid data leakage.here we are grouping given input prompt into particular folds.we group them based on the similarity of each prompt because if a prompt in the training fold has a similarity with the validation fold's prompt then we might end up with false non reliable training and validation accuracy which will further effect our model's training performance.here Out of fold predictions(OOF) are generated using training examples.In our project we used a separate kept dataset of 4004 samples for evaluation and 53473 samples used for finetuning.

Evaluation Metrics: The performance of the model was measured in different stages and using different metrics. Because our problem statement primarly focus on predicting probabilities we used logloss as our main evaluation metric .other metrics are also used to provide better understanding on how the model is able to give preference to a particular class.

1.MULTICLASS LOG LOSS: We have selected and used multiclass logloss as our main and primary metric because it is the specified metric that was informed to use in the Kaggle competition.this metric is used to measure how close the probabilities predicted by the model are to the actual class label.Here the lower the logloss the better the performance of the model will be.the main reason why we select logloss as our primary metric is also because it considers how confident the model is in its predictions.Model receives a large penalty when it assigns a high probability score to an incorrect prediction.this is our official leaderboard metric and it is public.

$$L = -\frac{1}{X}\sum_{i=1}^{X}\sum_{c=1}^{3} y_{ic}\log(z_{ic})$$

2.PRECISION: Precision is used to measure how many samples predicted by our model as a particular class belong to that particular class only.It is used to understand how reliable the model predictions are given when its predicts a particular category.in our project precision helps us to measure how often predictions of A wins or B wins or tie is actually correct .In the formula FP means false positives

and TP means true positives.

$$Precision = \frac{TP}{TP + FP}$$

3.ACCURACY: Accuracy measures the ratio of the samples for which the predicted class matches the actual label.it helps us to understand how the overall classification correctness is in the model.but one main drawback of the accuracy is that it does not consider the confidence of the predicted probabilities of a particular class.so even if the accuracy remains good but the model is bad with its confidence we might end up with bad results in the further predictions. We can safely use accuracy in this project because our dataset is balanced.because of these reason we use as an secondary supporting metric. X represents the total number of samples that were used.

$$Accuracy = \frac{\text{Number of Correct Predictions}}{X}$$

4.RECALL: Recall is used to measure how many actual samples that belongs to a particular class are correctly identifies by our model.it helps us to see whether the model is able to recognize the actual outcomes mainly when we are checking the 3 classes individually.here FN means false negatives.

$$Recall = \frac{TP}{TP + FN}$$

5.F1-SCORE: F1-Score calculates harmonic mean of precision and recall.we particularly use this because precision might miss many actual samples and recall might make a large number of incorrect prediction that is it does not consider false positives.so we combines both the metrics to measure both the type of classification metrics.higher the f1score better the balance between precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

6.PROBABILITY CALIBIRATION: Probability calibration is used to check if the confidence results that were predicted by the classifier correspond to the actual frequencies of the predicted results.here in this we use temperature scaling and tie class bias to adjust this confidence level and get better probability distributions.here we try to get as well calibrated model as we can which means given we predict 62

So we combine all these multiple metrics to check the performance of and compare them to see how well it might perform on the unseen new test data.

B. Quantitative and Qualitative Results

The results show that LightGBM got the best performance among the classical models woth an accracy of 49.03 and logloss of 1.0095.

TABLE II
CLASSICAL MODEL RESULTS

Classical model	OOF log loss ↓	Accuracy (%) ↑	Macro F1 ↑
MiniLM Logistic Regression	1.0783	41.30	0.3962
TF-IDF Logistic Regression	1.0208	48.19	0.4722
LightGBM	1.0095	49.03	0.4828
Classical ensemble	1.0075	49.22	0.4838

TABLE III
BASELINE COMPARISON

Method	Log loss ↓	Accuracy (%) ↑	Macro F1 ↑
Best individual model(LightGBM)	1.0095	49.03	0.4828
Uniform probability baseline	1.0986	34.91	0.1725
Classical ensemble	1.0075	49.22	0.4838

The classical ensemble got a logloss value of 1.0075 which means that model is learning useful patterns from the input and responses.

TABLE IV
LLM AND HYBRID ENSEMBLE RESULTS

Method	OOF logloss ↓	Accuracy(%) ↑	Macro F1 ↑
Classical ensemble	1.0032	49.63	0.4916
Qwen single-pass	1.0066	49.03	0.4855
Qwen with response-swap TTA	0.9947	50.07	0.4961
Qwen TTA + calibration	0.9940	49.93	0.4892
Final LLM + classical hybrid ensemble	0.9890	50.32	0.4982

The final hybrid model has given the best probability with lowest loss and highest accuracy performance among the evaluated types.

TABLE V
CLASSIFICATION REPORT BY OUTCOME CLASS

Class	Precision	Recall	F1-score	Support (samples)
Response A wins	0.5072	0.5509	0.5281	1347
Response B wins	0.5168	0.5521	0.5339	1420
Tie	0.4775	0.3953	0.4326	1237
Macro average	0.5005	0.4994	0.4982	4004

The lower recall value for tie indicates that identifying ties were more difficult than rest of the classes.
Figures and Visual Analysis:

C. Discussion and Limitations

Here when we were first trying out architecture for this project we tried MiniLM Embedding Logistic Regression but we got a loss of OOF log loss of 1.0783. we observed that because MiniLM has a drawback of taking only limited amount of text as input if the prompt exceeds the limit it will take only the limited amount of text

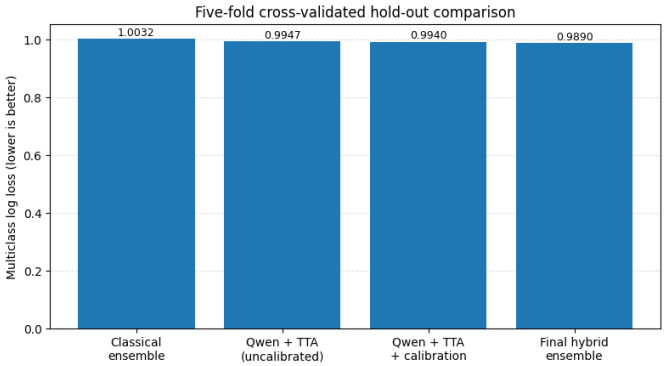


Fig. 2. Holdout logloss comparison

Inference:
we can see the reduction in logloss achieved by making improvements as we are changing the model which suggests that the performance of the model is increasing given we have achieved lowest loss of 0.9890 in final hybrid model

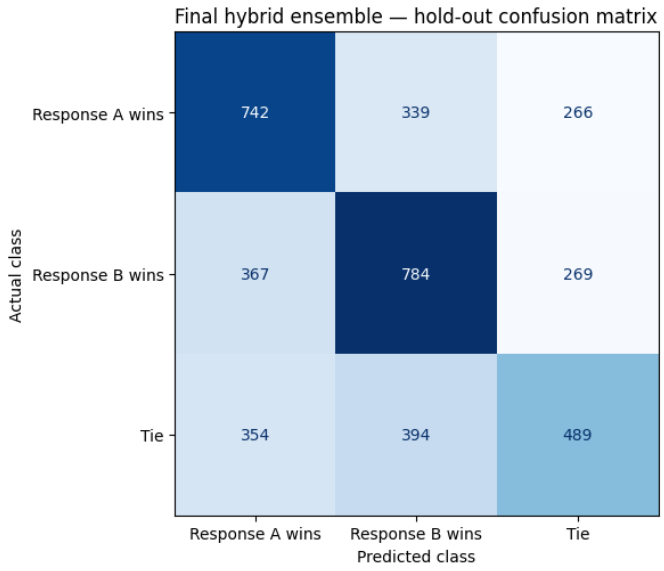


Fig. 3. Confusion Matrix of the LLM classifier

Inference:
We can observe from the diagonal of the confusion matrix that predicting tie is harder than rest of the responses because diagonal of this matrix represents the correct predictions which we can see that are 489 which are comparatively less than the rest of the classes.

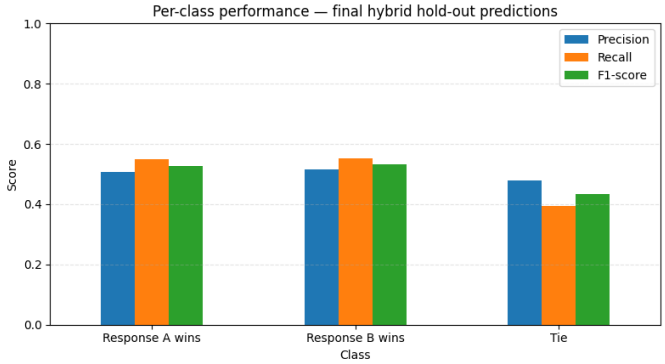


Fig. 4. Per-class evaluation results(precision recall and F1-score)

Inference:
We can see the comparison of the classification performance of A wins B wins and ties.we can clearly see that there is a drop in the recall value of the tie class stating that its kind off hard to predict while the remaining metrics are similar.

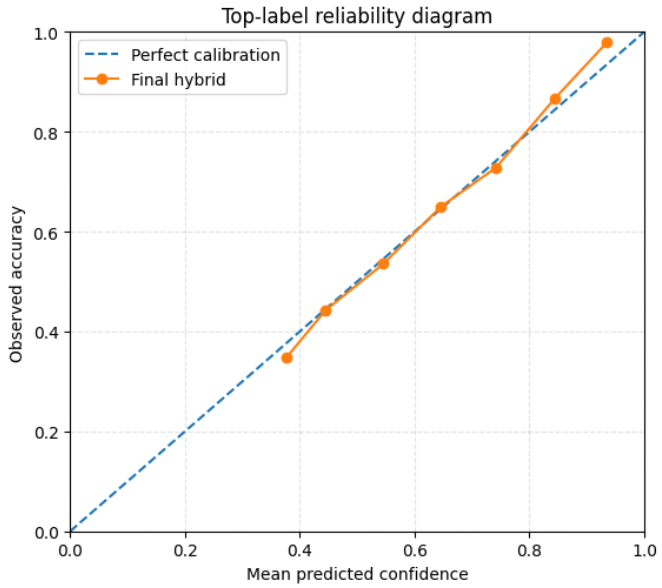


Fig. 5. Calibration Curve

Inference:
An increasing upward trend along the diagonal means that your Test Time Augmentation and temperature scaling steps are successful and we have almost well calibrated model.

missing the rest of the input prompt text which might lead to loss off the context or anything important present in the non read text which might eventually lead to a wrong prediction and probability distribution.we have also used temperature scaling because we observed form the original dataset input that most of the outcomes are favored towards tie so our might might overfit into thinking tie is the most favorable so we added the tie bias to normalize the model.we even added A/B swapping for data argumentation during test time to reduce the position bias.Also added 3fold StratifiedGroupKFold cross validation to reduce the risk of data leakage.after we have completed implementing all of this we have achieved a final log loss of 0.99361 on testing by kaggle.

V. Conclusion and Future Work

In this AI driven world knowing which response is preferred by the people is a main advantage to the users using AI in many of their daily activities like research,creating,ideation etc.The model needs to know what are the factors making a response very informative,interactive,accurate. So our main objective is to make the machine learning model learn differences between an human preferred response and a random response generated by the LLM.The responses we are comparing from the same model or different model.This a three class classification problem.So the the objective includes the model classifying between three classes(response a, response b, tie) with equal importance,not biased towards one class even though the training examples consists of 90 percent of tie.

Another part of our objective is to form a dual model which combines basic machine learning model with the fine tuned large language model so that basic ML model learns patterns between numerical features while the other path learns the relationships,significant words,phrases of the response so that it gives accurate probability values for each class through which we can classify.

Our dataset is balanced with no missing values with 57,477 training rows which contains prompt,response a, response b, model a, model b, three columns for a,b,tie in binary format.Most of the training examples are the examples o tie.

1.We have implemented one hot encoding that converts three binary comuns into one column containg values 0,1,2 for the three classes to reduce computaion.
2.We have removed model names from the training data to reduce preference bias based on the model.
3.We have identified position bias,verbosity bias,tie bias from the Exploratory data analysis we have performed.

We have extracted 218 numerical features from the from the prompts,responses. These features include
1.Prompt count(8)
2.Difference in length(15)
3.Difference in text quality(27)
4.Difference in formatting(21)
5.Difference in styles(12)
6.Prompt and response relevance(12)
7.Normalized differences(16)
8.Log ratios(21)
9.Relevance transforms(16)
10.Pair relations(6)
11.PCA(48)
12.prompt PCA(16)
Along with term frequency-inverse document frequency features. The basic component of our model consists of logistic regression model on TF-IDF features combined with lightGBM on numerical features. The deep learning model consists of two components where we fine tuned Qwen2.5 using Low rank adaption(LoRA).It trains only a small set of additional parameters rather than changing all the pretrained parameters that exist to our specific task.We introduced swap augmentation while testing to eradict position bias(model supports the first response more). The responses are exchanged and the predictions of the respective responses are reverted back into the original order and then calculated and merged with the actual result. We have merged the probabilities coming from two paths using weights which are hyperparameters so it knows how important both the paths are.The final calculated probability is calibrated using temperature scaling and tie bias so that predicted probability will be true to the actual probability distribution in the dataset.

We got a log loss of 0.99361 on testing by kaggle.(Present 65 on leader board out of 304 teams).

Limitations

1.The evaluation metric -log loss is unsuitable for this competition because this metric is used if the highly confident incorrect answers are not affordable which is fine here in this experiment.We need to encourage highly confident correct answers.Along with that,the responses are confined to user's preferences so it not suitable here.
2.The model is trained using the kaggle LLM dataset.Although we used cross validation it was not trained on various independent datasets or different applications so generalisation is a point of doubt here.

Future Improvements

1.The evaluation metric can be personalized to the particular user's preference rather than generalizing preferences.
2.Currently we are using only 27k samples so training the model one more epoch may improve the performance.

References

[1] H. Liu, "LLM-Classification-Fine-tuning," GitHub repository. [Online]. Available: https://github.com/henryliu513436/LLM-Classification-Fine-tuning. Accessed: Oct. 31, 2026.

[2] real-GBOY, "LLM-Classfication-1st," GitHub repository. [Online]. Available: https://github.com/real-GBOY/LLM-Classfication-1st. Accessed: Oct. 31, 2026.

[3] T-T312, "Kaggle_LLM_Comp," GitHub repository. [Online]. Available: https://github.com/T-T312/Kaggle_LLM_Comp. Accessed: Oct. 31, 2026.